

Documentação do Banco de Dados

Sistema de Gestão de E-mails, XMLs, Ordens de Coleta e API de Clientes

1. Objetivo do banco de dados

O banco de dados foi desenvolvido para centralizar as informações relacionadas a:

- Clientes e depositantes;
- E-mails recebidos;
- Arquivos XML anexados aos e-mails;
- Ordens de coleta;
- Motoristas e veículos;
- Resultado dos envios realizados pelo sistema;
- Tokens de autenticação da API;
- Histórico de acessos e requisições da API.

O banco utiliza o modelo relacional MySQL/MariaDB e foi preparado para funcionar em ambientes como XAMPP e hospedagem compartilhada da HostGator.

2. Visão geral das tabelas

Tabela	Finalidade
tb_clientes	Cadastro dos clientes e depositantes
tb_emails_recebidos	Registro dos e-mails processados
tb_arquivos_xml	Registro dos arquivos XML recebidos
tb_motoristas	Cadastro de motoristas e veículos
tb_ordens_coleta	Controle das ordens de coleta
tb_logs	Registro do resultado dos envios do sistema
tb_api_tokens	Controle dos tokens de acesso à API
tb_api_logs	Histórico de requisições realizadas na API

3. Tabela `tb_clientes`

Finalidade

Armazena os clientes ou depositantes utilizados pelo sistema.

Essa tabela é a base para os relacionamentos com e-mails, arquivos XML, motoristas e ordens de coleta.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único do cliente
<code>razao_social</code>	VARCHAR(200)	Sim	Razão social ou nome do cliente
<code>cnpj</code>	VARCHAR(14)	Sim	CNPJ armazenado somente com números
<code>email</code>	VARCHAR(255)	Não	E-mail relacionado ao cliente
<code>status</code>	ENUM	Sim	Situação do cliente: <code>ATIVO</code> ou <code>INATIVO</code>
<code>criado_em</code>	DATETIME	Sim	Data e hora de criação
<code>atualizado_em</code>	DATETIME	Sim	Data e hora da última atualização

Regras

- O CNPJ deve ser único.
- O CNPJ normalmente deve possuir 14 números.
- Pontos, barras e traços não devem ser armazenados.
- O cliente genérico `ERRO` utiliza o valor `0` no campo CNPJ.
- O status padrão é `ATIVO`.
- Clientes inativos não precisam ser excluídos; basta alterar o status para `INATIVO`.

Exemplo

```
Razão social: MONFIZA COMERCIO E IMPORTADORA LTDA
CNPJ: 22518066000100
E-mail: erros@ud.log.br
Status: ATIVO
```

4. Tabela `tb_emails_recebidos`

Finalidade

Registra os e-mails recebidos e processados pelo sistema.

Cada e-mail pode estar relacionado a um cliente e pode originar arquivos XML e ordens de coleta.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único do e-mail
<code>cliente_id</code>	BIGINT UNSIGNED	Não	Cliente identificado no e-mail
<code>recebido_em</code>	DATETIME	Sim	Data e hora do recebimento
<code>assunto</code>	VARCHAR(255)	Sim	Assunto original do e-mail
<code>remetente_email</code>	VARCHAR(255)	Sim	Endereço do remetente
<code>agendamento_em</code>	DATETIME	Não	Data e hora do agendamento identificado
<code>criado_em</code>	DATETIME	Sim	Data de inclusão no banco
<code>atualizado_em</code>	DATETIME	Sim	Data da última atualização

Chave estrangeira

```
tb_emails_recebidos.cliente_id  
→ tb_clientes.id
```

Relacionamento

- Um cliente pode possuir vários e-mails.
- Um e-mail pertence a, no máximo, um cliente.
- O campo `cliente_id` pode ficar vazio enquanto o sistema ainda não identificou o cliente.

Comportamento em exclusão

Caso o cliente seja excluído, o campo `cliente_id` do e-mail será definido como `NULL`.

O histórico do e-mail permanece armazenado.

5. Tabela `tb_arquivos_xml`

Finalidade

Registra os arquivos XML recebidos como anexos dos e-mails.

Cada XML fica vinculado ao e-mail de origem e ao cliente ou depositante identificado.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único do XML
<code>email_recebido_id</code>	BIGINT UNSIGNED	Sim	E-mail em que o XML foi recebido
<code>cliente_id</code>	BIGINT UNSIGNED	Sim	Cliente ou depositante do XML
<code>nome_arquivo</code>	VARCHAR(255)	Sim	Nome original do arquivo
<code>cfop</code>	VARCHAR(4)	Não	CFOP identificado no XML
<code>criado_em</code>	DATETIME	Sim	Data de criação do registro
<code>atualizado_em</code>	DATETIME	Sim	Data da última atualização

Chaves estrangeiras

```
tb_arquivos_xml.email_recebido_id  
→ tb_emails_recebidos.id
```

```
tb_arquivos_xml.cliente_id  
→ tb_clientes.id
```

Relacionamentos

- Um e-mail pode possuir vários arquivos XML.
- Um cliente pode possuir vários arquivos XML.
- Cada arquivo XML pertence a um único e-mail.
- Cada arquivo XML pertence a um único cliente.

Regras

- O CNPJ do depositante não é duplicado na tabela de XML.
- Para obter o CNPJ, deve-se consultar a tabela `tb_clientes` pelo campo `cliente_id`.
- O campo `cfop` aceita até quatro caracteres.
- Caso um e-mail seja excluído, seus arquivos XML também serão excluídos.

6. Tabela `tb_motoristas`

Finalidade

Armazena os motoristas e os dados dos veículos vinculados aos clientes.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único do motorista
<code>cliente_id</code>	BIGINT UNSIGNED	Sim	Cliente ao qual o motorista está vinculado
<code>nome_completo</code>	VARCHAR(150)	Sim	Nome completo do motorista
<code>cpf</code>	VARCHAR(11)	Não	CPF somente com números
<code>numero_cnh</code>	VARCHAR(20)	Não	Número da CNH
<code>validade_cnh</code>	DATE	Não	Data de validade da CNH
<code>placa_veiculo</code>	VARCHAR(10)	Não	Placa do veículo
<code>descricao_veiculo</code>	VARCHAR(150)	Não	Modelo ou descrição do veículo
<code>ano_veiculo</code>	SMALLINT UNSIGNED	Não	Ano do veículo
<code>criado_em</code>	DATETIME	Sim	Data de criação
<code>atualizado_em</code>	DATETIME	Sim	Data da última atualização

Chave estrangeira

```
tb_motoristas.cliente_id  
→ tb_clientes.id
```

Relacionamentos

- Um cliente pode possuir vários motoristas.
- Cada motorista está vinculado a um cliente.
- Um motorista pode aparecer em várias ordens de coleta.

Regras recomendadas

- Armazenar CPF somente com números.
- Armazenar a placa em letras maiúsculas.
- Não armazenar pontos ou traços no CPF.
- Validar a data de vencimento da CNH antes de permitir uma operação.

- O cadastro do motorista não deve ser excluído quando já possuir histórico operacional.

7. Tabela `tb_ordens_coleta`

Finalidade

Armazena as ordens de coleta identificadas nos e-mails ou cadastradas pelo sistema.

Permite registrar os dados previstos no agendamento e compará-los com o motorista efetivamente vinculado.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único da ordem
<code>numero_ordem_coleta</code>	VARCHAR(50)	Sim	Número da ordem de coleta
<code>cliente_id</code>	BIGINT UNSIGNED	Sim	Cliente responsável pela ordem
<code>motorista_id</code>	BIGINT UNSIGNED	Não	Motorista identificado ou apresentado
<code>email_recebido_id</code>	BIGINT UNSIGNED	Sim	E-mail de origem da ordem
<code>placa_prevista</code>	VARCHAR(10)	Não	Placa informada no agendamento
<code>motorista_nome_previsto</code>	VARCHAR(150)	Não	Nome previsto do motorista
<code>cnh_prevista</code>	VARCHAR(20)	Não	CNH prevista no agendamento
<code>criado_em</code>	DATETIME	Sim	Data de criação
<code>atualizado_em</code>	DATETIME	Sim	Data da última atualização

Chaves estrangeiras

```
tb_ordens_coleta.cliente_id  
→ tb_clientes.id
```

```
tb_ordens_coleta.motorista_id  
→ tb_motoristas.id
```

```
tb_ordens_coleta.email_recebido_id  
→ tb_emails_recebidos.id
```

Relacionamentos

- Um cliente pode possuir várias ordens de coleta.
- Um e-mail pode gerar uma ou mais ordens.
- Um motorista pode ser relacionado a várias ordens.
- Uma ordem pode ficar temporariamente sem motorista identificado.

Regra de duplicidade

A combinação abaixo deve ser única:

```
cliente_id + numero_ordem_coleta
```

Isso significa que o mesmo cliente não pode possuir duas ordens com o mesmo número.

Clientes diferentes podem possuir ordens com números iguais.

Dados previstos e dados efetivos

Os campos abaixo representam os dados informados no agendamento:

```
placa_prevista  
motorista_nome_previsto  
cnh_prevista
```

O campo abaixo representa o motorista identificado no cadastro oficial:

```
motorista_id
```

Essa separação permite comparar o agendamento com a apresentação real.

8. Tabela `tb_logs`

Finalidade

Registra o resultado dos envios realizados pelo sistema.

Essa tabela pode ser utilizada para registrar, por exemplo:

- E-mails enviados com sucesso;
- E-mails que não foram enviados;
- Falhas de processamento;
- Mensagens operacionais relacionadas aos envios.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único do log
<code>descricao</code>	TEXT	Sim	Descrição detalhada do evento
<code>status</code>	ENUM	Sim	Resultado: <code>ENVIADO</code> ou <code>NAO_ENVIADO</code>
<code>criado_em</code>	DATETIME	Sim	Data e hora automática do registro

Exemplo de sucesso

Descrição: E-mail da ordem 12345 enviado para o cliente MONFIZA.
Status: ENVIADO

Exemplo de falha

Descrição: Não foi possível enviar o e-mail da ordem 12345.
Status: NAO_ENVIADO

Observação

A tabela `tb_logs` registra eventos funcionais do processo de envio.

Ela é diferente da tabela `tb_api_logs`, que registra acessos e chamadas realizadas na API.

9. Tabela `tb_api_tokens`

Finalidade

Armazena os tokens utilizados para autenticar as integrações que acessam a API.

Exemplos de integrações:

- n8n;
- Make;

- Sistemas internos;
- Aplicações web;
- Scripts de integração;
- Serviços externos autorizados.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único do token
<code>nome</code>	VARCHAR(100)	Sim	Nome da integração ou finalidade
<code>token_prefixo</code>	VARCHAR(20)	Sim	Parte inicial do token para identificação
<code>token_hash</code>	CHAR(64)	Sim	Hash SHA-256 do token
<code>ativo</code>	TINYINT(1)	Sim	Define se o token está ativo
<code>expira_em</code>	DATETIME	Não	Data e hora de expiração
<code>ultimo_uso_em</code>	DATETIME	Não	Última utilização do token
<code>criado_em</code>	DATETIME	Sim	Data de criação
<code>atualizado_em</code>	DATETIME	Sim	Data da última atualização

Regras

- O token completo não é armazenado no banco.
- O banco armazena somente o hash SHA-256.
- O token completo é exibido somente no momento da criação.
- O campo `ativo` utiliza:
 - `1` para token ativo;
 - `0` para token desativado.
- O campo `expira_em` pode ficar vazio para tokens sem expiração definida.
- Um token expirado ou inativo não pode acessar os endpoints protegidos.

Exemplo

Nome: n8n producao
 Ativo: 1
 Expira em: 2027-12-31 23:59:59

Desativação de token

```
UPDATE tb_api_tokens
SET ativo = 0
WHERE id = 1;
```

Segurança

Não é possível recuperar o token original pelo valor armazenado no banco.

Caso o token seja perdido, deve-se criar um novo token e desativar o anterior.

10. Tabela `tb_api_logs`

Finalidade

Registra as requisições realizadas na API.

Essa tabela é utilizada para auditoria, segurança, identificação de falhas e análise de utilização.

Estrutura

Campo	Tipo	Obrigatório	Descrição
<code>id</code>	BIGINT UNSIGNED	Sim	Identificador único do acesso
<code>token_id</code>	BIGINT UNSIGNED	Não	Token utilizado na requisição
<code>metodo</code>	VARCHAR(10)	Sim	Método HTTP utilizado
<code>rota</code>	VARCHAR(500)	Sim	Endpoint acessado
<code>ip</code>	VARCHAR(45)	Não	Endereço IP de origem
<code>user_agent</code>	VARCHAR(500)	Não	Aplicação ou navegador utilizado
<code>status_http</code>	SMALLINT UNSIGNED	Sim	Código HTTP retornado
<code>criado_em</code>	DATETIME	Sim	Data e hora da requisição

Chave estrangeira

```
tb_api_logs.token_id  
→ tb_api_tokens.id
```

Exemplos de métodos

```
GET  
POST  
PUT  
PATCH
```

DELETE
OPTIONS

Na versão atual da API de clientes, o principal método utilizado é o `GET`.

Exemplos de códigos HTTP

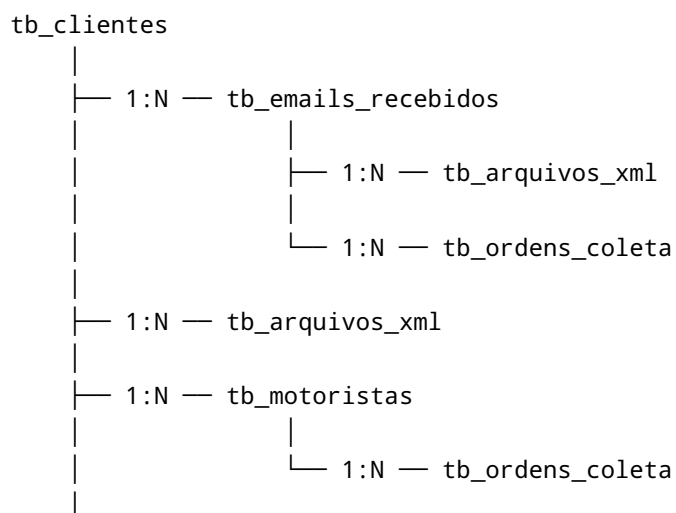
Código	Significado
200	Consulta realizada com sucesso
204	Requisição processada sem conteúdo
400	Parâmetro inválido
401	Token ausente ou inválido
403	Acesso não autorizado
404	Cliente ou rota não encontrada
405	Método HTTP não permitido
429	Limite de requisições excedido
500	Erro interno da API

Comportamento em exclusão

Caso um token seja excluído, o campo `token_id` dos registros de acesso será definido como `NULL`.

O histórico da requisição será preservado.

11. Diagrama simplificado dos relacionamentos



```

└─ 1:N — tb_ordens_coleta

tb_api_tokens
└─ 1:N — tb_api_logs

tb_logs
└─ Não possui chave estrangeira

```

12. Resumo das chaves estrangeiras

Tabela de origem	Campo	Tabela de destino	Campo
tb_emails_recebidos	cliente_id	tb_clientes	id
tb_arquivos_xml	email_recebido_id	tb_emails_recebidos	id
tb_arquivos_xml	cliente_id	tb_clientes	id
tb_motoristas	cliente_id	tb_clientes	id
tb_ordens_coleta	cliente_id	tb_clientes	id
tb_ordens_coleta	motorista_id	tb_motoristas	id
tb_ordens_coleta	email_recebido_id	tb_emails_recebidos	id
tb_api_logs	token_id	tb_api_tokens	id

13. Fluxo operacional do sistema

Etapas 1 — Recebimento do e-mail

O sistema recebe um e-mail e registra:

- Cliente identificado;
- Data e hora de recebimento;
- Assunto;
- Remetente;
- Data e hora do agendamento.

Os dados são armazenados em:

```
tb_emails_recebidos
```

Etapa 2 — Identificação do cliente

O cliente pode ser identificado por:

- CNPJ encontrado no XML;
- Nome ou razão social;
- Endereço do remetente;
- Conteúdo do assunto;
- Regras internas do sistema.

O cliente identificado é relacionado pelo campo:

cliente_id

Caso nenhum cliente seja identificado, pode ser utilizado o cadastro genérico:

ERRO

Etapa 3 — Processamento dos XMLs

Cada arquivo XML anexado é registrado separadamente.

São armazenados:

- E-mail de origem;
- Cliente ou depositante;
- Nome do arquivo;
- CFOP.

Os dados são gravados em:

tb_arquivos_xml

Etapa 4 — Criação da ordem de coleta

O sistema identifica o número da ordem e registra:

- Cliente;
- E-mail de origem;
- Motorista, quando localizado;
- Placa prevista;
- Nome previsto;

- CNH prevista.

Os dados são gravados em:

tb_ordens_coleta

Etapa 5 — Identificação do motorista

O sistema pode procurar o motorista por:

- CPF;
- Número da CNH;
- Placa;
- Nome;
- Cliente relacionado.

Quando localizado, o campo abaixo é preenchido:

motorista_id

Etapa 6 — Envio e registro do resultado

Após o processamento, o sistema pode enviar o resultado por e-mail ou para outra integração.

O resultado é salvo em:

tb_logs

Com um dos status:

ENVIADO
NAO_ENVIADO

Etapa 7 — Consulta pela API

Uma integração faz uma requisição para a API utilizando:

Authorization: Bearer TOKEN

A API:

1. Recebe o token;
2. Gera o hash SHA-256;
3. Procura o hash em `tb_api_tokens` ;
4. Verifica se o token está ativo;
5. Verifica se o token expirou;
6. Executa a consulta;
7. Registra a requisição em `tb_api_logs` .

14. Endpoints atuais da API

Verificação da API

`GET /health`

Não exige token.

Listar clientes

`GET /api/v1/clientes`

Exige Bearer Token.

Filtros disponíveis

nome
cnpj
email
status
pagina
por_pagina

Exemplo

`GET /api/v1/clientes?status=ATIVO&pagina=1&por_pagina=20`

Buscar cliente pelo ID

```
GET /api/v1/clientes/{id}
```

Exemplo:

```
GET /api/v1/clientes/1
```

Buscar cliente pelo CNPJ

```
GET /api/v1/clientes/cnpj/{cnpj}
```

Exemplo formatado:

```
GET /api/v1/clientes/cnpj/02.271.463/0001-13
```

Exemplo sem formatação:

```
GET /api/v1/clientes/cnpj/02271463000113
```

A API remove automaticamente pontos, barras e traços antes da consulta.

15. Consultas SQL úteis

Listar clientes ativos

```
SELECT
    id,
    razao_social,
    cnpj,
    email,
    status
FROM tb_clientes
WHERE status = 'ATIVO'
ORDER BY razao_social;
```

Buscar cliente pelo CNPJ

```
SELECT
    id,
    razao_social,
    cnpj,
    email,
    status
FROM tb_clientes
WHERE cnpj = '02271463000113'
LIMIT 1;
```

Consultar os últimos logs de envio

```
SELECT
    id,
    descricao,
    status,
    criado_em
FROM tb_logs
ORDER BY criado_em DESC
LIMIT 100;
```

Consultar tokens ativos

```
SELECT
    id,
    nome,
    token_prefixo,
    ativo,
    expira_em,
    ultimo_uso_em,
    criado_em
FROM tb_api_tokens
WHERE ativo = 1
ORDER BY id DESC;
```

Consultar os últimos acessos à API

```
SELECT
    l.id,
    l.metodo,
    l.rota,
    l.ip,
    l.status_http,
```

```
l.criado_em,  
t.nome AS token_nome  
FROM tb_api_logs l  
LEFT JOIN tb_api_tokens t  
ON t.id = l.token_id  
ORDER BY l.id DESC  
LIMIT 100;
```

Consultar quantidade de acessos por token

```
SELECT  
t.id,  
t.nome,  
COUNT(l.id) AS total_requisicoes,  
MAX(l.criado_em) AS ultimo_acesso  
FROM tb_api_tokens t  
LEFT JOIN tb_api_logs l  
ON l.token_id = t.id  
GROUP BY  
t.id,  
t.nome  
ORDER BY total_requisicoes DESC;
```

16. Regras de segurança

- O arquivo `.env` não deve ser disponibilizado publicamente.
- O `.env` não deve ser enviado ao GitHub.
- A API deve ser utilizada somente com HTTPS.
- O ambiente de produção deve utilizar `APP_DEBUG=false`.
- Cada sistema ou integração deve possuir seu próprio token.
- Tokens que não são mais utilizados devem ser desativados.
- Tokens temporários devem possuir data de expiração.
- O token completo deve ser guardado em local seguro.
- A criação de token pelo navegador deve permanecer desativada após o primeiro uso.
- O usuário do banco deve possuir apenas os privilégios necessários.
- Senhas do banco nunca devem ser gravadas diretamente nos arquivos PHP.

17. Configuração na HostGator

Na hospedagem compartilhada, o banco deve ser criado pelo cPanel.

O usuário MySQL normalmente não possui permissão para executar:

CREATE DATABASE

O processo correto é:

1. Criar o banco em **Bancos de Dados MySQL** no cPanel;
2. Criar ou selecionar um usuário MySQL;
3. Vincular o usuário ao banco;
4. Marcar todos os privilégios necessários;
5. Abrir o phpMyAdmin;
6. Selecionar o banco criado;
7. Importar somente as tabelas.

Os comandos abaixo não devem ser incluídos no arquivo importado pela HostGator:

```
CREATE DATABASE db_gestao_coletas;  
USE db_gestao_coletas;
```

O nome real do banco normalmente possui o prefixo da conta:

```
usuario_db_gestao_coletas
```

Exemplo de configuração no `.env`:

```
DB_HOST=localhost  
DB_PORT=3306  
DB_DATABASE=usuario_db_gestao_coletas  
DB_USERNAME=usuario_banco  
DB_PASSWORD=senha_do_usuario_mysql
```

18. Diferença entre os dois tipos de log

```
tb_logs
```

Registra eventos funcionais do sistema:

```
E-mail enviado  
E-mail não enviado  
Falha de processamento  
Resultado de uma ordem
```

tb_api_logs

Registra acessos técnicos à API:

Método HTTP
Rota acessada
IP
Token utilizado
Código HTTP
Data e hora

As duas tabelas possuem funções diferentes e devem ser mantidas separadas.

19. Considerações finais

A estrutura atual permite:

- Centralizar o cadastro dos clientes;
- Relacionar e-mails aos clientes;
- Processar vários XMLs por e-mail;
- Criar várias ordens a partir de um e-mail;
- Manter o cadastro de motoristas;
- Comparar dados previstos e efetivamente apresentados;
- Registrar sucesso ou falha nos envios;
- Disponibilizar os clientes por uma API protegida;
- Controlar tokens individualmente;
- Auditar todos os acessos à API.

O modelo foi estruturado para evitar duplicação de informações e manter o histórico das operações.